DEVIXO SOLUTIONS

AI/ML Internship Program

Task 03

End-to-End Machine Learning Project

Student Name: Muhammad Ali

Internship Domain: AI/ML

Submission Date: October 25, 2026

Project: Used-Car Price Prediction (regression) on the CarDekho Vehicle Dataset

1. Introduction

This report documents Task 03 of the Devixo Solutions AI/ML Internship Program: a complete machine learning project from data preparation to a saved model and a prediction interface. It covers feature engineering and selection, model development, hyperparameter tuning with cross-validation, feature importance, saving the best model with Joblib and Pickle, and a bonus Streamlit app.

The dataset lists used cars with their age, kilometres driven, fuel, transmission, owner history, engine, power, mileage and name. The problem is a **regression** problem: predict the selling price of a car. (Accuracy, precision and recall from Task 02 do not apply to a numeric target; error measures such as MAE, RMSE, RMSLE and R2 are used instead.)

The next table links each part of the Task 03 sheet to the section of this report that answers it.

Task 03 requirement	Report section
Part 1: import, missing values, duplicates, feature engineering, feature selection, scaling	6, 9, 10, 11
Part 2: at least four models (Random Forest, SVM, Gradient Boosting, Decision Tree; XGBoost optional)	12
Part 3: hyperparameter tuning, cross-validation, feature importance	13, 16
Part 4: save the best model (Joblib and Pickle)	17
Part 5 documentation: dataset, workflow, model selection, results, limitations, future improvements	3-9, 14-15, 19, 20
Bonus: Streamlit prediction interface	18

2. Objective

The objective is to build a clean, reproducible machine learning pipeline on a real-world dataset: prepare the data without leaking test information, compare at least four algorithms fairly, tune them with cross-validation, choose a model by evidence, explain what it learned, and save it so that it can be reused.

3. Dataset Name

Vehicle dataset from CarDekho, file `Car details v3.csv` (stored in the project as `data/raw/car_details_v3.csv`).

4. Dataset Source

Kaggle: https://www.kaggle.com/datasets/nehalbirla/vehicle-dataset-from-cardekho. The rows are used-car listings from the CarDekho website (India). The newest car is from 2020, so the listings are probably from around that year or earlier and the findings describe that market and period. The price column has no stated currency; it is assumed to be Indian rupees. The licence should be checked on the Kaggle page before the data is reused.

5. Dataset Description

Each row is one used car. The columns describe the car (name, manufacturing year, fuel, transmission), its use (kilometres driven, number of owners, seller type), its specification (mileage, engine size, maximum power, torque, number of seats) and the selling price, which is the target.

6. Dataset Size

	Raw dataset	After preprocessing
Rows (cars)	8,128	6,926
Columns	13	13 (9 of them are used by the model)
Exact duplicate rows	1,202 (14.8%)	0 (removed)
Rows with a missing value in mileage, engine, max_power, seats or torque	222	209 (about 3.0%; imputed inside the pipeline)
Train / test rows	-	5,540 / 1,386 (80% / 20%)
File size	about 1.04 MB	-

The task sheet requires at least 3,000 rows. After removing duplicates the dataset still has **6,926** rows, so the requirement is met.

7. Features/Columns Description

Group	Columns	Meaning
Identity	name	Brand, model and variant in one text (for example 'Maruti Swift Dzire VDI'). Too many distinct values to use directly, so the brand is extracted.
Age and use	year, km_driven, owner	Manufacturing year, kilometres driven, and number of previous owners (First Owner to Test Drive Car)
Type	fuel, transmission, seller_type	Diesel, Petrol, CNG or LPG; Manual or Automatic; Individual, Dealer or Trustmark Dealer
Specification	mileage, engine, max_power, torque, seats	Stored as text with units ('23.4 kmpl', '1248 CC', '74 bhp'); torque is free text with mixed units
Target	selling_price	Price of the car (assumed rupees); heavily right-skewed

8. Methodology

The work was done in Python 3.12.3 with Pandas 3.0.2, NumPy 2.4.4, Scikit-learn 1.8.0, XGBoost 3.4.1, Matplotlib and Seaborn in Jupyter notebooks, with reusable code in `src/`. The random seed is 42 everywhere. The workflow is:

- **Notebook 01 - Data audit and EDA:** read-only inspection of the raw file; records every data problem and the decision it leads to.
- **Notebook 02 - Preparation and feature work:** remove duplicates, split 80/20 **before** learning anything from the data, parse the text columns, engineer and select features, scale.
- **Notebook 03 - Modelling:** compare five models and two reference baselines with 5-fold cross-validation, tune with `RandomizedSearchCV`, choose the best by cross-validation, evaluate once on the test set, explain the model, save it.
- **One pipeline:** feature engineering, imputation, scaling, encoding, the model and the log-price conversion form one scikit-learn pipeline. Only training data is used to fit it, so the test set cannot leak, and the saved model accepts a raw car description.
- **Quality checks:** 33 automated tests (`pytest`) check the parsing code, the data split, the saved model and the Streamlit app.

9. Part 1 - Data Preparation

Every action is recorded below with its reason. The raw file was never changed; all work was done on copies.

Step	Action	Affected	Reason
1	Loaded the CSV	8,128 rows, 13 columns	The task requires at least 3,000 rows.
2	Removed exact duplicate rows	1,202 rows	Identical in all 13 columns, including exact price and kilometres. Done **before** the split so that a listing cannot be in both train and test. Assumption: they are repeated listings (the file has no listing ID).
3	Split 80% / 20% (`random_state=42`)	5,540 train / 1,386 test	Done before any statistic is learned. Median price is 400,000 in train and 409,999 in test. The test set is not used again until the final evaluation.
4	Parsed `mileage`, `engine`, `max_power` from text to numbers	3 columns	Models need numbers. Zero mileage (15 rows) and a blank power value (1 row in the clean data) are not real measurements, so they become missing.
5	Filled missing numbers with the median (inside the pipeline, fitted on train only)	mileage 184, engine 172, max_power 173 (train)	About 3% of rows, missing together (205 rows have all five columns missing), so they cannot be rebuilt from each other and dropping them would waste data. Medians learned from training data: mileage 19.4, engine 1,248, max_power 81.83.
6	Engineered `car_age`, `brand`, `log_km_driven`	3 new features	Explained and tested in section 11.
7	Selected 9 features, dropped `seats`, `seller_type`, `torque`, `name`, `year`, `km_driven`	see section 11	Chosen by experiment, not by guess.
8	Scaled the 5 numeric features with `StandardScaler`	5 columns	Needed by SVR; harmless for trees. Fitted on training data only, inside the pipeline.
9	One-hot encoded `brand`, `fuel`, `transmission`, `owner`	31 new columns	Categories with fewer than 10 training cars are grouped as 'infrequent' (11 of 30 brands). Unseen categories are handled safely. After preprocessing the training matrix has 36 columns.
10	Trained on log(price) and converted predictions back to rupees	target	Price is very skewed; checked by experiment in section 11.

Result: 6,926 clean rows (the requirement of at least 3,000 is met) and a single pipeline that turns a raw car description into a model-ready row without using the test set.

10. Exploratory Data Analysis

Statistics of the numeric columns (cleaned data, all rows):

Variable	Mean	Median	Std. dev.	Min	Max
year	2,013	2,014	4	1,983	2,020
km_driven	73,996	70,000	58,358	1	2,360,457
selling_price	517,271	400,000	519,767	29,999	10,000,000

Fuel: Diesel (3,755), Petrol (3,077), CNG (56), LPG (38). Transmission: Manual (6,342), Automatic (584). Mileage is in km/kg for 87 cars, all of them CNG or LPG cars (CNG 52, LPG 35), and in kmpl for the rest.

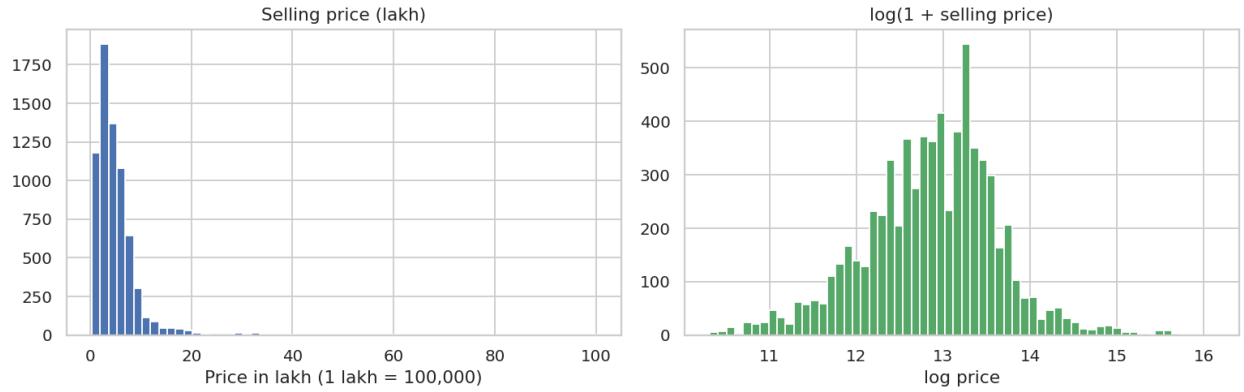


Figure 10.1: the price is very skewed (skewness 5.57); the log scale is much more symmetric (skewness -0.16)

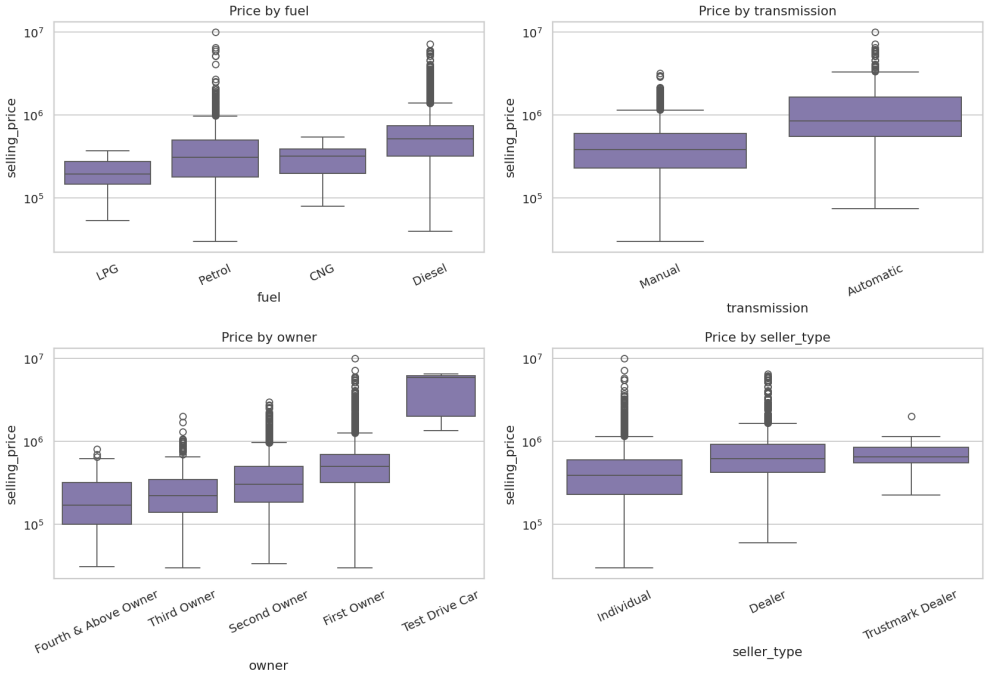


Figure 10.2: price by fuel, transmission, owner and seller type (log scale)

The median price is 850,000 for automatic and 385,000 for manual cars, 520,000 for diesel and 310,000 for petrol, and 500,000 for first owners against 170,000 for fourth owners and above. Rank correlations with log price: car age -0.71, maximum power 0.63, engine 0.48, seats 0.32, kilometres -0.29, mileage 0.02. These are associations, not causes, and several features move together (for example power and engine size).

Suspicious values. Two cars show more than one million kilometres (the highest is 2,360,457 km on a 2007 car), which looks like a typing error. They were kept: there are only two, and the log transform limits their influence. The highest price (10,000,000 for a Volvo XC90 T8) is plausible for a luxury car and was not treated as an error.

11. Feature Engineering and Feature Selection

Feature engineering. `car_age` is 2020 minus the year; `brand` is the first word of the name ('Land' is corrected to 'Land Rover'); `log_km_driven` is the logarithm of the kilometres (skewness 5.02 before, -1.39 after). Two more were tried: `km_per_year` and `bhp_per_litre`. Every choice was tested with 5-fold cross-validation on the training set; the score is RMSLE (error on the log-price scale, roughly a typical percentage error; lower is better) and the fold-to-fold noise is about 0.01.

Experiment (Random Forest, 5-fold CV)	RMSLE	R2	MAE
A. parsed numbers, no brand	0.2408	0.8790	82,906
B. A + brand	0.2301	0.8835	80,994
C. B, log(km) instead of km	0.2302	0.8825	80,993
D. C + km_per_year + bhp_per_litre	0.2300	0.8804	81,096

- **Brand helps:** the error falls from 0.2408 to 0.2301 (about one fold-noise standard deviation, and consistent with the drop-one test below).
- **Log kilometres and the two extra features made no measurable difference for a Random Forest** (trees ignore monotonic transforms). `log_km_driven` is kept because the skew is a known problem for SVR; `km_per_year` and `bhp_per_litre` are **not used** because they did not help.
- **Log target:** RMSLE 0.2302 with log(price) against 0.2341 without, and R2 0.8825 against 0.8831. The effect is small for a forest; the log target is kept because it matches the relative-error metric and suits SVR.

Feature selection. Three checks on training data. (1) Correlation: only two pairs of numeric features reach 0.6 (engine and max_power 0.68; engine and seats 0.66), so no feature is removed for redundancy alone. (2) Mutual information (a screening score for each feature alone) was computed, but it measures each feature in isolation, so it ranks car age lower than the better test below. (3) Drop-one-feature test: remove one feature at a time and measure how much the error rises.

Feature removed	RMSLE	Increase vs all 11 features
(none - all 11)	0.2302	+0.0000
car_age	0.3068	+0.0765
log_km_driven	0.2330	+0.0027
mileage	0.2357	+0.0055
engine	0.2319	+0.0017
max_power	0.2419	+0.0116
seats	0.2305	+0.0003
brand	0.2409	+0.0107
fuel	0.2318	+0.0016
seller_type	0.2304	+0.0002
transmission	0.2310	+0.0007
owner	0.2330	+0.0028

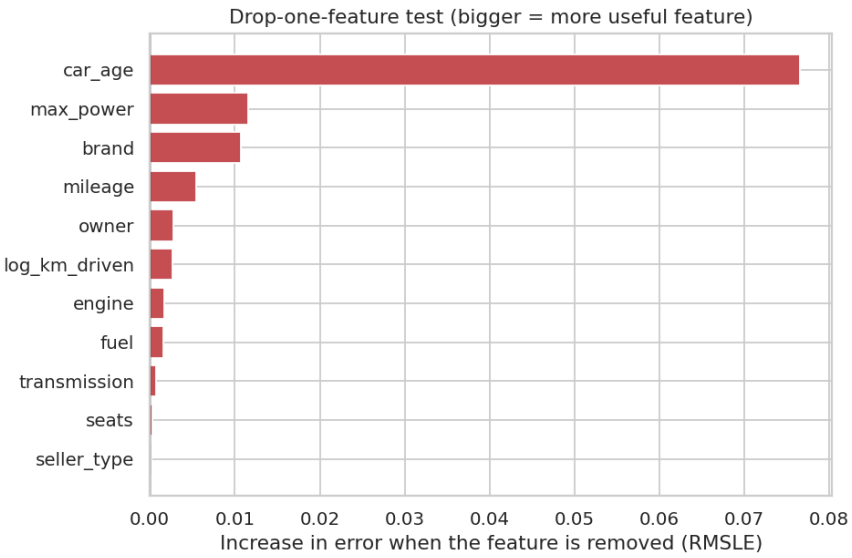


Figure 11.1: increase in cross-validated error when each feature is removed (bigger bar = more useful feature)

`seats` (+0.0003) and `seller_type` (+0.0002) have almost no effect. Removing both together was checked on three model types:

Model	RMSLE all 11	RMSLE reduced to 9	Difference	Fold noise (std)
Random Forest	0.2302	0.2305	+0.0003	0.0107
Gradient Boosting	0.2345	0.2342	-0.0002	0.0088
SVR	0.2296	0.2305	+0.0009	0.0133

All differences are far smaller than the fold noise, so both features were dropped. `fuel`, `transmission` and `engine` have small effects too, but they were kept: the removal threshold was strict, and they have an obvious link to price. **Final features (9):** `car_age`, `log_km_driven`, `mileage`, `engine`, `max_power` (numeric) and `brand`, `fuel`, `transmission`, `owner` (categorical). `torque` was not used because it is free text with mixed units.

12. Part 2 - Model Development

Every model is placed in the **same pipeline** (feature engineering, imputation, scaling, encoding, log target) and trained on the same training set, so the comparison is fair. The first five rows of the next table are the required models plus the optional XGBoost; Ridge regression and an 'always predict the median price' model are added as reference points.

Model	Role	Needs scaling?	Default settings used
Decision Tree	Required	No	defaults (no depth limit)
Random Forest	Required	No	200 trees
SVR (Support Vector Machine)	Required	Yes	defaults (RBF kernel)
Gradient Boosting	Required	No	defaults
XGBoost	Optional in the task	No	defaults (`hist` method)
Ridge regression	Reference	Yes	`alpha=1`
Always predict the median	Reference floor	-	-

Metrics. RMSLE is the selection metric: prices run from about 30 thousand to 10 million, and RMSLE measures relative error, so a 10% miss on a cheap and on an expensive car count the same. R2 is the

share of price variation explained; MAE and RMSE are in rupees (RMSE punishes large misses and is dominated by expensive cars). MedAPE is the median absolute percentage error.

Cross-validation results with default settings (training set, 5 folds):

Model	CV RMSLE	CV std	CV R2	CV MAE	Fit time (s)
XGBoost (optional)	0.2284	0.0108	0.8742	79,319	0.351
SVR (Support Vector Machine)	0.2295	0.0122	0.8337	82,506	0.858
Random Forest	0.2300	0.0104	0.8840	80,464	3.273
Gradient Boosting	0.2342	0.0091	0.8688	87,347	0.736
Ridge regression (reference)	0.2626	0.0136	0.8655	97,883	0.076
Decision Tree	0.2990	0.0108	0.7899	103,835	0.134
Baseline (always median price)	0.7678	0.0182	-0.0520	282,423	0.141

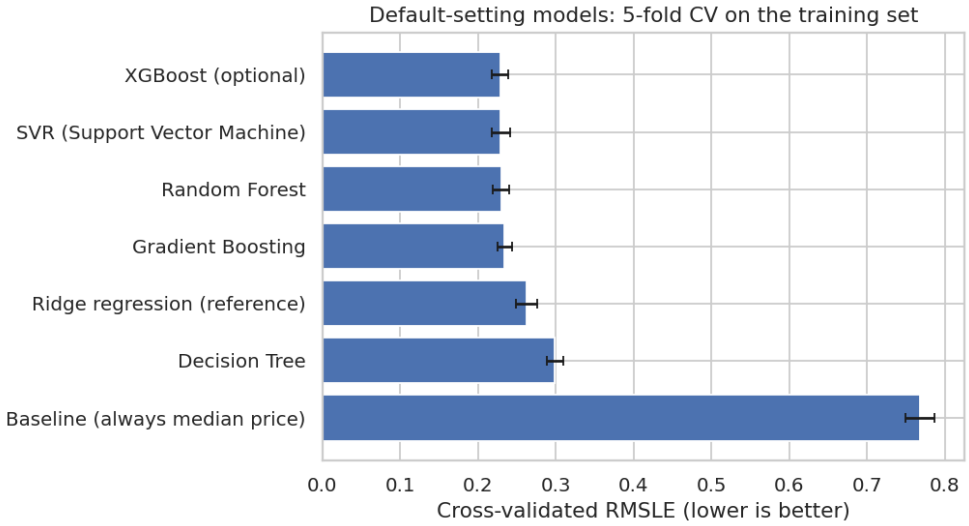


Figure 12.1: default-setting models, 5-fold cross-validated RMSLE (bars show the spread over folds)

- The always-median model has RMSLE 0.7678 and R2 -0.0520, so the metrics are wired correctly (R2 near 0 is expected).
- All tree-based models and SVR beat Ridge regression (0.2626); the best default models (XGBoost, SVR, Random Forest) are within about 0.002 of each other, which is far less than the fold noise.
- The unpruned Decision Tree is the weakest real model (0.2990); it overfits when it has no depth limit.

13. Part 3 - Model Optimization (Hyperparameter Tuning and Cross-Validation)

Design. `RandomizedSearchCV` tries 15 random combinations of settings per model. Every combination is scored with **5-fold cross-validation on the training set only**: the training data is cut into five parts, the model is trained on four and scored on the fifth, five times, and the scores are averaged. The score is RMSLE. Random search was used instead of a full grid because it covers wide ranges in much less time. After the search each model is refitted on the whole training set with its best settings. The test set plays no part in tuning.

13.1 Settings searched

Model	Parameters searched (15 random combinations each)
Decision Tree	`max_depth` {4, 6, 8, 10, 12, 15, None}; `min_samples_leaf` {1, 2, 5, 10, 20}; `min_samples_split` {2, 5, 10, 20}
Random Forest	`n_estimators` {100, 200, 300}; `max_depth` {None, 10, 15, 20}; `min_samples_leaf` {1, 2, 4}; `max_features` {sqrt, 0.5, 1.0}
Gradient Boosting	`n_estimators` {100, 200, 300, 400}; `learning_rate` {0.03, 0.05, 0.1, 0.2}; `max_depth` {2, 3, 4, 5}; `subsample` {0.7, 0.85, 1.0}; `min_samples_leaf` {1, 5, 10}
SVR (SVM)	`C` {1, 3, 10, 30, 100}; `epsilon` {0.01, 0.05, 0.1, 0.2}; `gamma` {scale, 0.01, 0.03, 0.1}
XGBoost	`n_estimators` {200, 400, 600}; `learning_rate` {0.03, 0.05, 0.1}; `max_depth` {3, 4, 5, 6}; `subsample` {0.7, 0.85, 1.0}; `colsample_bytree` {0.6, 0.8, 1.0}; `min_child_weight` {1, 3, 5}

13.2 Results

Model	Best parameters found	CV RMSLE	CV std	CV R2	CV MAE	Search (s)
Decision Tree	{min_samples_split: 2, min_samples_leaf: 10, max_depth: None}	0.2623	0.0087	0.8375	94,892	12
Random Forest	{n_estimators: 300, min_samples_leaf: 2, max_features: 0.5, max_depth: 20}	0.2227	0.0105	0.8769	80,046	146
SVR (SVM)	{gamma: scale, epsilon: 0.05, C: 1}	0.2299	0.0125	0.8338	82,124	259
Gradient Boosting	{subsample: 1.0, n_estimators: 300, min_samples_leaf: 5, max_depth: 4, learning_rate: 0.1}	0.2209	0.0108	0.8843	78,691	140
XGBoost (optional)	{subsample: 0.85, n_estimators: 600, min_child_weight: 3, max_depth: 5, learning_rate: 0.03, colsample_bytree: 0.6}	0.2178	0.0113	0.8825	77,570	31

Model	Default CV RMSLE	Tuned CV RMSLE	Improvement
Decision Tree	0.2990	0.2623	+0.0367
Random Forest	0.2300	0.2227	+0.0073
SVR (SVM)	0.2295	0.2299	-0.0004
Gradient Boosting	0.2342	0.2209	+0.0133
XGBoost (optional)	0.2284	0.2178	+0.0106



Figure 13.1: cross-validated RMSLE before and after tuning (re-drawn from the notebook's numbers)

- **Tuning helped every tree-based model** (improvements 0.0073 to 0.0367); the Decision Tree gained most because its depth limit was tuned.

- **Tuning did not help SVR** (-0.0004). Its best C is 1, the smallest value offered, so a wider range (smaller C) might do better.
- All tuned tree-based models (0.2178 to 0.2227) beat Ridge regression (0.2626), so the extra complexity is justified.
- 15 random combinations is a modest search, and several winning values lie inside large grids, so a longer search might change small differences between models.

14. Model Selection and Final Test Results

Selection rule, fixed before the test set was used: the model with the lowest tuned cross-validated RMSLE wins. The code applies it. **XGBoost** was selected (CV RMSLE 0.2178, fold spread 0.0113). Its settings: {subsample: 0.85, n_estimators: 600, min_child_weight: 3, max_depth: 5, learning_rate: 0.03, colsample_bytree: 0.6}.

It is effectively a tie. The runner-up, Gradient Boosting, is only 0.0031 behind, much less than the fold-to-fold noise (0.0113). Changing the choice after seeing test scores would be selecting a model with the test set, so the rule was kept.

The test set (1,386 cars) was then used once. All tuned models are shown for transparency; the choice above was made from cross-validation alone.

Model	R2	MAE (rupees)	RMSE (rupees)	RMSLE	Median % error
XGBoost (optional) (selected)	0.9275	71,361	126,069	0.2153	12.3%
Gradient Boosting	0.9401	69,992	114,649	0.2164	12.6%
Random Forest	0.9225	72,025	130,394	0.2250	12.6%
SVR (SVM)	0.8544	76,556	178,726	0.2279	12.7%
Decision Tree	0.8987	84,069	149,089	0.2547	13.9%
Always predict the median	-0.0460	259,547	478,963	0.7488	42.9%

- Selected model on the test set: median error **12.3%** of the price, MAE **71,361**, RMSLE **0.2153** (always guessing the median: 0.7488), R2 0.9275.
- Gradient Boosting scores slightly higher on test R2 (0.9401) and MAE (69,992), the selected model on RMSLE: consistent with a tie.
- **Why test R2 (0.9275) is higher than cross-validation R2 (0.8825):** R2 is dominated by a few very expensive cars, and the test set happens to contain only 6 cars above 30 lakh, while the training set has 43 (a fair share would be about 11). RMSLE hardly moves (0.2178 in cross-validation, 0.2153 on test), so **RMSLE and the median percentage error are the more trustworthy summaries**.

15. Error Analysis of the Selected Model

A single score can hide where a model fails, so the errors were examined by price band and one by one.

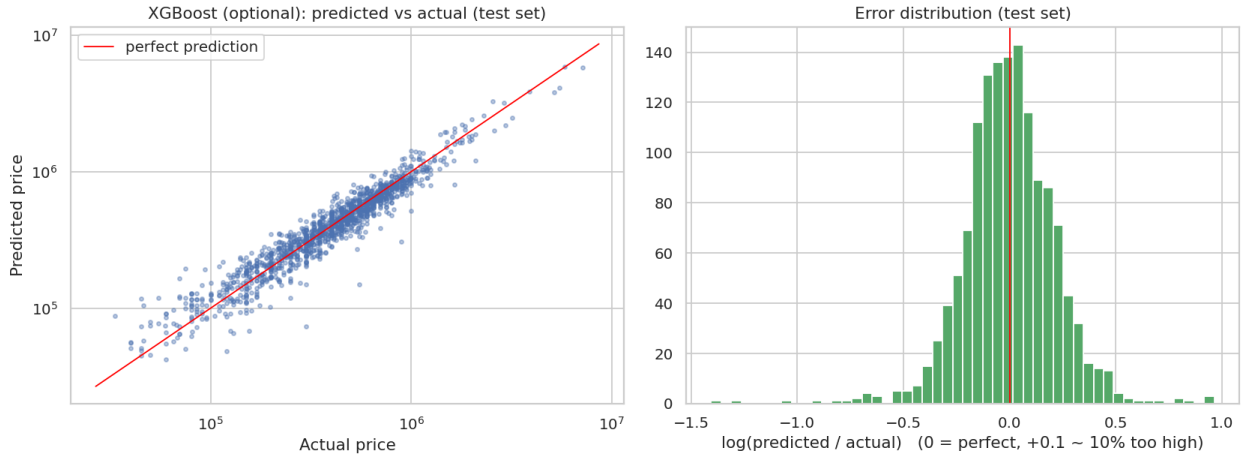


Figure 15.1: left: predicted against actual price on the test set (log scales, red line = perfect); right: distribution of the errors, centred near zero

Actual price band	Cars	Median % error	Median signed % error	Mean abs. error (rupees)
< 2 lakh	276	19.0%	+11.1%	30,743
2-4 lakh	412	12.3%	+2.5%	45,846
4-6 lakh	332	11.5%	-1.9%	68,817
6-10 lakh	286	9.2%	-5.4%	90,551
> 10 lakh	80	13.6%	-8.0%	284,859

1 lakh = 100,000 rupees. A positive signed error means the model predicted too high.

- The model is least accurate for **cars under 2 lakh** (median error 19.0%) and tends to over-price them; it **under-prices cars above 10 lakh** (median -8.0%), which is typical of a model that pulls extreme values toward the average.
- Of the 10 largest percentage errors, 9 are over-estimates of cheap, old cars (for example Maruti Wagon R VXI: actual 33,351, predicted about 87,388); the other is Maruti Gypsy King Soft Top (actual 300,000, predicted about 73,583).
- An indicative price range of the predicted price x 0.78 to x 1.27 (the middle 80% of out-of-fold errors on training data) covered **79.9%** of the untouched test cars.

16. Feature Importance

Permutation importance (main result). One input column of the test set is shuffled at a time, and the rise in error is measured (10 repeats). It works for any model and measures real predictive value on unseen data.

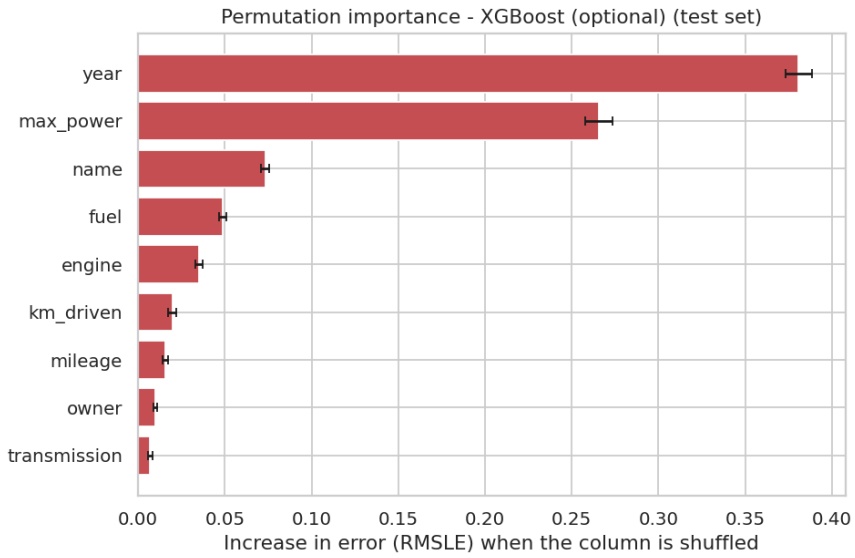


Figure 16.1: permutation importance; `name` *carries the brand and* `year` *carries the car age*

Input column	Importance (RMSLE increase)	Spread (std)
year	0.3808	0.0075
max_power	0.2657	0.0081
name	0.0733	0.0023
fuel	0.0488	0.0022
engine	0.0351	0.0020
km_driven	0.0197	0.0022
mileage	0.0159	0.0016
owner	0.0098	0.0012
transmission	0.0068	0.0013

Car age (`year`, 0.381) and maximum power (0.266) dominate, followed by brand (0.073), fuel (0.049) and engine size (0.035). This agrees with the drop-one test in section 11.

Built-in importance of the XGBoost model gives a different order (brand 0.26, car_age 0.16, max_power 0.15, fuel 0.14, ...): it is summed over one-hot columns and favours features with many splits, so the permutation result is treated as the main one. Importance shows what the model **used**, not what **causes** a price; correlated features (power and engine) share credit.

17. Part 4 - Save the Best Model (Joblib and Pickle)

The **whole pipeline** is saved (feature engineering, imputation, scaling, encoding, model and the log-price conversion), so the saved file accepts a raw car description and returns a price.

File	Size	Purpose
`models/best_model.joblib`	0.40 MB	Joblib format (compressed)
`models/best_model.pkl`	1.40 MB	Pickle format
`models/model_metadata.json`	small	Model name, settings, metrics, input ranges, library versions

Verification (never trust a save that was not reloaded):

- Both files reload and give predictions identical to the in-memory model on the whole test set.

- A brand-new Python process loads both files and produces identical predictions.
- The model handles missing specifications and a brand it has never seen (a Tesla, absent from training) and still returns a price.
- Saved with scikit-learn 1.8.0 and XGBoost 3.4.1; `requirements.txt` pins the versions because a saved model can only be loaded safely with the same versions. Pickle files must only be loaded from trusted sources.
- **Recovery path (found when the model was first run on another computer):** with different library versions, loading failed with `XGBoostError: input stream corrupted`. `python -m src.retrain` now rebuilds the model in seconds from the saved best settings, using the libraries installed on that computer, and prints its test scores next to the original ones (they matched exactly when tested). The app also explains the problem and both fixes on screen instead of crashing.

18. Bonus - Streamlit Prediction Interface

`app.py` loads the saved Joblib pipeline and lets a user enter a car's brand, year, kilometres, fuel, transmission, owner, mileage, engine size and power. It builds one raw-format row and calls `predict`. It shows the estimated price and a likely range (predicted price x 0.78 to x 1.27), warns when an input is outside the range seen in training (for example very high kilometres), and explains its limits in an 'About this model' box. Run it with `streamlit run app.py`.

The app is covered by automated tests (Streamlit's headless test runner starts the app, clicks the button and checks the result and the warnings) and was started on a real server to check that it loads. No screenshot is included in this report.

19. Conclusion and Recommendation

I built a complete, leak-free pipeline on 6,926 used-car listings: removed 1,202 duplicate rows, split the data before learning anything from it, parsed and engineered features, chose features by experiment, compared four required models plus XGBoost with cross-validation, tuned them, selected one by a rule fixed in advance, evaluated it once on the test set, and saved it in two formats.

Recommendation: use the tuned XGBoost model (or, equivalently, Gradient Boosting, which is statistically tied). On the untouched test set it has a median error of 12.3% of the price and an RMSLE of 0.2153, against 0.7488 for always guessing the median. It is most reliable for mid-priced cars (2 to 10 lakh) and should be used with the indicative price range, not as a point valuation.

From this project I learned that the split must come before any fitting, that one pipeline prevents leakage and makes saving simple, that features should be kept or dropped on evidence, that R2 can mislead when a few expensive items dominate, that two models can be tied and the honest answer is to say so, and that a saved model must be reloaded and tested.

20. Limitations

- **Old market data:** the newest car is from 2020 and the prices are listing prices from that period, so the model is **not valid for current prices**.
- **Uneven accuracy:** weaker for very cheap old cars and for luxury cars (only 80 test cars are above 10 lakh).
- **Rare categories** (Test Drive Car owners, CNG and LPG fuel, 11 rare brands) have too few rows to learn from and are grouped as 'infrequent'.

- **Duplicate removal is an assumption**, and near-duplicate listings (same model, similar details) may remain on both sides of the split, which could make scores slightly optimistic.
- **Random split, not a time split:** it does not test performance on future years.
- **Unused columns:** `torque`, `seats` and `seller_type` were not used.
- **Modest tuning:** 15 random combinations per model; SVR's best value was at the edge of its range. A single train/test split also leaves some noise in the test scores.
- **Currency and licence:** the currency is assumed to be rupees, and the dataset's licence was not verified.

21. Future Improvements

- Parse `torque` into Nm and test it; extract the car **model** (not only the brand) as a feature.
- Use newer data and a time-aware split to measure performance on future years.
- Group near-identical listings together during cross-validation to reduce leakage.
- Run a wider or Bayesian hyperparameter search; try quantile models for a principled price range.
- Deploy the app (for example on Streamlit Community Cloud) and monitor incoming inputs against the training ranges.

22. Major Findings

- After removing 1,202 duplicates the dataset has 6,926 cars; the price is very skewed (skewness 5.57) and about 3% of rows lack mileage, engine, power and seats together.
- Car age is by far the most useful feature (error rises 0.0765 without it), then maximum power and brand; `seats` and `seller_type` add nothing measurable.
- Tuning improved every tree-based model, but not SVR. XGBoost (optional) and Gradient Boosting are tied (0.0031 apart in cross-validation, noise 0.0113).
- Test result of the selected model: median error 12.3%, MAE 71,361, RMSLE 0.2153.
- Errors are largest for cheap old cars (over-priced) and for luxury cars (under-priced).
- The saved Joblib and Pickle models reload to identical predictions, and the Streamlit app works on top of them.

23. Appendix - Key Code

The code below is copied from the project files (text, not screenshots). The notebooks in the repository contain all cells with their saved outputs.

A.1 Feature-engineering transformer (src/car_features.py)

```
"""
Reusable feature-engineering code for the used-car price project.

Why this file exists
--------------------
The raw CSV stores numbers as text ("23.4 kmpl", "1248 CC", "74 bhp") and the
car `name` contains the brand. We need the SAME cleaning to happen
  (a) when we train the model, and
  (b) when the Streamlit app predicts for a new car.
Putting it in ONE transformer inside the saved scikit-learn Pipeline guarantees
both places behave identically, and it means the saved model accepts RAW input.

The transformer is "stateless": it learns nothing from the data, so it cannot
leak information from the test set. Anything that DOES learn from data (median
imputation, scaling, one-hot categories) lives in the ColumnTransformer that
comes after it and is fitted on training data only.
"""
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

# The newest car in the dataset is from 2020, so we measure "age" from 2020.
# (Any constant works for tree models; we fix it and document it.)
REFERENCE_YEAR = 2020

# Raw dataset columns we read (everything except the target and `torque`,
# which we deliberately do not use - see notebook 02).
RAW_INPUT_COLUMNS = [
    "name", "year", "km_driven", "fuel", "seller_type",
    "transmission", "owner", "mileage", "engine", "max_power", "seats",
]
# `seats` and `seller_type` were tested in notebook 02 and did not improve
# the model, so the final model does not use them. They are OPTIONAL inputs:
# if they are missing the transformer fills them with NaN instead of failing.
OPTIONAL_INPUT_COLUMNS = ["seller_type", "seats"]
REQUIRED_INPUT_COLUMNS = [c for c in RAW_INPUT_COLUMNS if c not in OPTIONAL_INPUT_COLUMNS]

# Columns the transformer OUTPUTS (a superset of what the model finally uses).
ENGINEERED_NUMERIC = [
    "car_age", "km_driven", "log_km_driven", "km_per_year",
    "mileage", "engine", "max_power", "seats", "bhp_per_litre",
]
ENGINEERED_CATEGORICAL = ["brand", "fuel", "seller_type", "transmission", "owner"]
ALL_OUTPUT_COLUMNS = ENGINEERED_NUMERIC + ENGINEERED_CATEGORICAL

# Brands whose name has two words, so "first word" alone would be wrong.
_TWO_WORD_BRANDS = {"Land": "Land Rover"}

_NUMBER_AT_START = r"^\s*(\d+(?:\.\d+)?)"


def _leading_number(series: pd.Series) -> pd.Series:
    """'23.4 kmpl' -> 23.4 ; '1248 CC' -> 1248.0 ; ' bhp' -> NaN ; 74 -> 74.0"""
    text = series.astype("string")
    return pd.to_numeric(text.str.extract(_NUMBER_AT_START)[0], errors="coerce").astype(float)


def extract_brand(name: pd.Series) -> pd.Series:
    """'Maruti Swift Dzire VDI' -> 'Maruti' ; 'Land Rover Discovery' -> 'Land Rover'"""
    first = name.astype("string").str.strip().str.split().str[0]
    return first.replace(_TWO_WORD_BRANDS).astype(object)


class CarFeatureEngineer(BaseEstimator, TransformerMixin):
    """Turn raw car rows into clean numeric + categorical columns."""

    def fit(self, X, y=None):
        self._check_columns(X)
        self.n_features_in_ = len(REQUIRED_INPUT_COLUMNS)
        return self

    @staticmethod
    def _check_columns(X):
        if not isinstance(X, pd.DataFrame):
            raise TypeError("CarFeatureEngineer expects a pandas DataFrame with raw car columns.")
        missing = [c for c in REQUIRED_INPUT_COLUMNS if c not in X.columns]
        if missing:
            raise ValueError(f"Missing required input columns: {missing}")

    def transform(self, X):
        self._check_columns(X)
        out = pd.DataFrame(index=X.index)

        year = pd.to_numeric(X["year"], errors="coerce").astype(float)
        km = pd.to_numeric(X["km_driven"], errors="coerce").astype(float)
        km = km.where(km >= 0)                      # negative km is impossible -> missing

        out["car_age"] = (REFERENCE_YEAR - year).clip(lower=0)
        out["km_driven"] = km
        out["log_km_driven"] = np.log1p(km)
        out["km_per_year"] = km / out["car_age"].clip(lower=1)

        # Text -> number. Zero or blank values are not real measurements -> NaN.
```

```
        out["mileage"] = _leading_number(X["mileage"]).where(lambda s: s > 0)
        out["engine"] = _leading_number(X["engine"]).where(lambda s: s > 0)
        out["max_power"] = _leading_number(X["max_power"]).where(lambda s: s > 0)
        seats = X["seats"] if "seats" in X.columns else pd.Series(np.nan, index=X.index)
        out["seats"] = pd.to_numeric(seats, errors="coerce").astype(float)
        out["bhp_per_litre"] = out["max_power"] / (out["engine"] / 1000.0)

        out["brand"] = extract_brand(X["name"])
        for col in ["fuel", "seller_type", "transmission", "owner"]:
            values = X[col] if col in X.columns else pd.Series(np.nan, index=X.index)
            out[col] = values.astype("string").str.strip().astype(object)

        return out[ALL_OUTPUT_COLUMNS]

    def get_feature_names_out(self, input_features=None):
        return np.array(ALL_OUTPUT_COLUMNS, dtype=object)
```

Listing A.1: parsing and feature engineering, shared by training and the app

A.2 Preprocessing and pipeline (src/modeling.py)

```
def build_preprocessor(numeric_features=None, categorical_features=None):
    """Imputation + scaling + one-hot encoding. Fitted on training data only."""
    numeric_features = NUMERIC_FEATURES if numeric_features is None else numeric_features
    categorical_features = CATEGORICAL_FEATURES if categorical_features is None else categorical_features

    numeric_steps = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),          # needed by SVR; harmless for trees
    ])
    categorical_steps = Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(
            handle_unknown="infrequent_if_exist",
            min_frequency=MIN_CATEGORY_FREQUENCY,
            sparse_output=False,
        )),
    ])
    transformers = [("num", numeric_steps, list(numeric_features))]
    if categorical_features:
        transformers.append(("cat", categorical_steps, list(categorical_features)))
    return ColumnTransformer(transformers, remainder="drop", verbose_feature_names_out=False)


def build_pipeline(model, numeric_features=None, categorical_features=None, log_target=True):
    """Full raw-input -> price pipeline around any scikit-learn regressor."""
    inner = Pipeline([
        ("features", CarFeatureEngineer()),
        ("preprocess", build_preprocessor(numeric_features, categorical_features)),
        ("model", model),
    ])
    if not log_target:
        return inner
    return TransformedTargetRegressor(regressor=inner, func=np.log1p, inverse_func=np.expm1)
```

Listing A.2: the common pipeline used for every model

A.3 Hyperparameter tuning (notebook 03)

```
P = "regressor__model__"       # prefix that reaches the model inside our pipeline
N_ITER = 15

search_spaces = {
    "Decision Tree": (DecisionTreeRegressor(random_state=RANDOM_STATE), {
        P + "max_depth": [4, 6, 8, 10, 12, 15, None],
        P + "min_samples_leaf": [1, 2, 5, 10, 20],
        P + "min_samples_split": [2, 5, 10, 20],
    }),
    "Random Forest": (RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=-1), {
        P + "n_estimators": [100, 200, 300],
        P + "max_depth": [None, 10, 15, 20],
        P + "min_samples_leaf": [1, 2, 4],
        P + "max_features": ["sqrt", 0.5, 1.0],
    }),
    "Gradient Boosting": (GradientBoostingRegressor(random_state=RANDOM_STATE), {
        P + "n_estimators": [100, 200, 300, 400],
        P + "learning_rate": [0.03, 0.05, 0.1, 0.2],
        P + "max_depth": [2, 3, 4, 5],
        P + "subsample": [0.7, 0.85, 1.0],
        P + "min_samples_leaf": [1, 5, 10],
    }),
    "SVR (Support Vector Machine)": (SVR(), {
        P + "C": [1, 3, 10, 30, 100],
        P + "epsilon": [0.01, 0.05, 0.1, 0.2],
        P + "gamma": ["scale", 0.01, 0.03, 0.1],
    }),
}
if HAVE_XGB:
    search_spaces["XGBoost (optional)"] = (XGBRegressor(random_state=RANDOM_STATE, n_jobs=-1, tree_method="hist"), {
        P + "n_estimators": [200, 400, 600],
        P + "learning_rate": [0.03, 0.05, 0.1],
        P + "max_depth": [3, 4, 5, 6],
        P + "subsample": [0.7, 0.85, 1.0],
        P + "colsample_bytree": [0.6, 0.8, 1.0],
        P + "min_child_weight": [1, 3, 5],
    })

tuning_scoring = {"rmsle": "neg_root_mean_squared_log_error", "r2": "r2", "mae": "neg_mean_absolute_error"}
searches, tuned_cv, best_params = {}, {}, {}

for name, (model, space) in search_spaces.items():
    t0 = time.perf_counter()
    s = RandomizedSearchCV(build_pipeline(model), space, n_iter=N_ITER, cv=cv, scoring=tuning_scoring,
                           refit="rmsle", random_state=RANDOM_STATE, n_jobs=1)
    s.fit(X_train, y_train)
    i = s.best_index_
    searches[name] = s
    tuned_cv[name] = {"RMSLE": -s.cv_results_["mean_test_rmsle"][i], "RMSLE_std": s.cv_results_["std_test_rmsle"][i],
                      "R2": s.cv_results_["mean_test_r2"][i], "MAE": -s.cv_results_["mean_test_mae"][i]}
    best_params[name] = {k.replace(P, ""): (v.item() if hasattr(v, "item") else v) for k, v in s.best_params_.items()}
    print(f"{name:30s} tuned CV RMSLE {tuned_cv[name]['RMSLE']:.4f} | {time.perf_counter() - t0:.0f}s")
```

Listing A.3: RandomizedSearchCV for every model

A.4 Saving the model (notebook 03)

```
brand_counts = pd.Series(
    X_train["name"].str.split().str[0].replace({"Land": "Land Rover"})).value_counts()
metadata = {
    "project": "Used-car price prediction (Devixo Solutions AI/ML Internship, Task 03)",
    "created": date.today().isoformat(),
    "selected_model": best_name,
    "best_hyperparameters": best_params[best_name],
    "selection_rule": "lowest 5-fold cross-validated RMSLE on the training set after RandomizedSearchCV",
    "cv_metrics_train": {k: float(v) for k, v in tuned_table.loc[best_name].items()},
    "test_metrics": {k: float(v) for k, v in test_table.loc[best_name].items()},
    "indicative_range_ratio": {"low": float(q10), "high": float(q90), "test_coverage": float(inside)},
    "features": {"numeric": NUMERIC_FEATURES, "categorical": CATEGORICAL_FEATURES,
                 "required_input_columns": REQUIRED_INPUT_COLUMNS},
    "training_rows": int(len(X_train)), "test_rows": int(len(X_test)),
    "currency_note": "Prices are in the dataset's currency (assumed Indian rupees; CarDekho listings). Newest car year is 2020.",
    "input_choices": {
        "brand": [b for b, n in brand_counts.items() if n >= 10],
        "fuel": sorted(X_train["fuel"].dropna().unique().tolist()),
        "transmission": sorted(X_train["transmission"].dropna().unique().tolist()),
        "owner": X_train["owner"].value_counts().index.tolist(),
    },
    "input_ranges": {
        "year": [int(X_train["year"].min()), int(X_train["year"].max())],
        "km_driven_p99": float(X_train["km_driven"].quantile(0.99)),
        "mileage_p01_p99": [float(v) for v in
            X_train["mileage"].str.extract(r"^\s*(\d+(?:\.\d+)?)")[0].astype(float).quantile([0.01, 0.99])],
        "engine_cc_p01_p99": [float(v) for v in
            X_train["engine"].str.extract(r"^\s*(\d+(?:\.\d+)?)")[0].astype(float).quantile([0.01, 0.99])],
        "max_power_bhp_p01_p99": [float(v) for v in
            X_train["max_power"].str.extract(r"^\s*(\d+(?:\.\d+)?)")[0].astype(float).quantile([0.01, 0.99])],
    },
    "library_versions": {"python": platform.python_version(), "scikit-learn": sklearn.__version__,
                         "pandas": pd.__version__, "numpy": np.__version__, "joblib": joblib.__version__,
                         **({"xgboost": __import__("xgboost").__version__} if HAVE_XGB else {})},
}

joblib_path = MODEL_DIR / "best_model.joblib"
pickle_path = MODEL_DIR / "best_model.pkl"
joblib.dump(best_model, joblib_path, compress=3)
with open(pickle_path, "wb") as f:
    pickle.dump(best_model, f)
with open(MODEL_DIR / "model_metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)

for p in [joblib_path, pickle_path, MODEL_DIR / "model_metadata.json"]:
    print(f"{p.name:22s} {p.stat().st_size / 1e6:8.2f} MB")
```

Listing A.4: metadata, Joblib and Pickle

24. GitHub Repository Link

https://github.com/Al7bh/Devixo-Internship (folder: `Task_03_ML`)

The task folder contains the three executed notebooks, `src/` (reusable code), the data, the saved models in `models/`, the figures and result tables in `reports/`, the tests, `app.py`, this report and its builder script in `report/`, `requirements.txt` and the README.